

パワポは古いよ、もう捨てよう

WEBブラウザでプレゼン

2025年8月29日

§ 1. 目標、進め方

本日の目標

このプレゼンテーションを通じて

- テキストからスライドが作成できるようになる
- Markdownの書式を用いてコンテンツを作成できる
- CSSでデザインを調整できるようになる

具体的な内容

- 準備 → 環境構築とツールインストール
- 基本操作 → 最初のスライド作成
- コンテンツ作成 → MarkdownとHTML
- デザイン → CSSで見た目調整
- 応用 → 便利に使うアイデア

細々した利点など

利点(主にパワポに比べて)

コンテンツもデザインもテキストファイルで完結する

- テキストエディタだけで作成できる

コンテンツとデザインが分離している

- 一度デザインを決めたらもう気にしなくていい
- 制限があると逆に思いつきに引きずられなくなる

AIや自動化と相性がいい

- AIの生成物をテキストに絞れる(デザイン不要)
- テキストなので開発物と同様に自動化可能

以下の知識、技術が身に付く

- HTML, CSS, Markdown, JavaScript
- 使い方による: Node.js, Python, Docker

一応、デメリットも

- テキストとスライド出力の差をイメージする想像力が必要
- テキスト入力とスライド表示にタイムラグがある

対象ユーザー

- パワポ資料の特にデザインが面倒くさいと思う人

関連情報

ページの右上には関連情報へのリンクを載せています。アイコンの意味は次の通りです

-  : 書籍系
-  : オンラインの情報
-  : ↑ のうち、特に勉強になったもの
-  : 間接的な参考情報
-  : プログラミング関係

§ 2. 環境構築とツール導入

全体の位置付けと使用ファイル

位置付け

- **準備:環境構築とツールインストール** ← 今ココ
- **基本操作:最初のスライド作成** ← これも
- コンテンツ作成:MarkdownとHTML
- デザイン:CSSで見た目調整
- 応用:便利に使うアイデア

スクリプト&サンプルファイル置き場

このあと利用するスクリプトやサンプルファイルは[こちらのレポジトリ](#)に置いてあります。ご自由にダウンロードください

必要なツールの導入

新規に導入するツールは `Marp CLI` 1つです

Marp CLIのダウンロード

[marp-cliのGitHubリポジトリ](#)からOSに合わせた `marp-cli-*`をダウンロードします

- Windowsの場合: `marp-cli-v4.2.1-win.zip`
- Macの場合: `marp-cli-v4.2.1-mac.tar.gz`
- Linuxの場合: `marp-cli-v4.2.1-linux.tar.gz`

※バージョン名は最新版に合わせて読み替えてください

Marp CLIのインストール

ダウンロードしたファイルを解凍し、中のファイル (`marp.exe` または `marp`) を適当なフォルダにコピーします。フォルダは `PATH` が通っていると使いやすいです(通ってなくても本レクは完了できます)

テキストエディタの準備

`marp` はテキストファイルを読み込んでスライドに変換します。よってテキストエディタが必須です

各OSに標準で付いているテキストエディタ(メモ帳など)で構いません。しかし、これを機に便利なエディタを取り入れるのであれば下記を推奨します

-  [Visual Studio Code](#) およびその派生
-  [Vim](#) (習得難易度は高いですが超強力です)
-  [Neovim](#) (同上。Vimより少し現代的です)

サンプルファイルの用意

サンプルのダウンロード

このあと使用するサンプルファイルは[こちらのレポジトリ](#)からダウンロードください

フォルダ構成

次のように数字を付けたフォルダに分けています

```
sample/  
├── 01/  
│   └── slide.md  
├── 02/  
│   └── slide.md  
└── 03/  
    ├── css/  
    └── slide.md
```

各トピックの冒頭で使用するフォルダを明示しています

例) 01 フォルダ内のファイルを使用します

実行準備

先ほどダウンロードした `marp` は `PATH` が通ったフォルダか、もしくは指定されたサンプルフォルダに移動・コピーしてください。(Mac、UNIX系の場合は実行権付与も)

これにより、各トピックにて説明しているコマンドやスクリプトが動作するようになります

動作確認と初回スライド作成

動作確認

コマンドラインで以下を実行:

```
marp --version
```

出力例: `@marp-team/marp-cli v4.2.1`

⊗ エラーが出た場合は以下をチェック

1. PATHが通っているか
 2. カレントディレクトリ内に`marp`があるか
 3. 実行権が付与されているか
 4. OSが合っているか
- を確認してください

Hello Worldスライド作成

01 フォルダ内のファイルを使用します

次のコマンドを実行します

```
marp hello.md -o output.html
```

`output.html` が作成されるので、開いて確認してみましょう

Hello Marp!

これが最初のスライドです。

このようなページが表示されればOKです

スライド作成ツールの導入は以上です
次章からは実際にスライドを作成しながら
基本的な書式を説明します

§ 3. スライド作成の基本書式

サンプルファイルをエディタで開く

スライド生成の確認

02 フォルダ内のファイルを使用します

次のコマンドを実行します

```
marp intro.md -o output.html
```

`output.html` が作成されるので開いて確認します

私の最初のプレゼン

作成者：あなたの名前

作成日：2025年7月31日

このようなページが表示されればOKです
左右のカーソルキーでページが切り替えられます

テキストエディタで開きます

先ほどの `intro.md` という名前のファイルをテキストエディタで開きます。

`.md` は見慣れない拡張子かも知ですが、中身は普通のテキストファイルです。ただし、見出しや箇条書きといったテキストの構造を明示する記法が定義されています（このファイルフォーマットを **Markdown** と呼びます）

i 次ページの説明は `intro.md` の内容に沿っています

冒頭の設定(Front Matter)

冒頭部分

intro.md

```
—  
marp: true  
theme: default  
size: 16:9  
—
```

intro.md の先頭は `—` で始まり、`—` で終わるブロックになっています。中身の各行は `項目名:値` の形式で、Marpにおける基本設定を記述します

- `marp: true` はMarp有効化の宣言です
- `theme: default` はビジュアルテーマの指定です
- `size: 16:9` はスライドサイズの指定です

このブロックはとりあえず書いておけばOKです

✎ この様な設定の書き方をFrontMatterと呼びます

テーマの変更

intro.md の `theme:default` を `theme:gaia` に書き換えて保存します。次いで下記コマンドを実行します

```
marp intro.md -o output.html
```

output.html が作成されるので開いて確認します
下記のようにビジュアルが変更されます

私の最初のプレゼン

作成者：あなたの名前

作成日：2025年7月31日

スライド本文

スライド本文を試してみる

Front Matter以降がスライド本文になります

intro.md

私の最初のプレゼン

作成者：あなたの名前
作成日：2025年7月31日

自己紹介

- 名前：ピカチュウ
- 所属：任天堂
- 趣味：**10まんボルト** ⚡ で世界を照らす

一言メッセージ

ピカピカ～！ ✨

本文はMarkdown記法を使って書かれています

Markdown記法とは

Markdown記法は基本的にテキスト文章そのままです

ただし、一部特定の書き方については、それに対応する意味(構造)を表すものとして解釈されます。この書き方と意味との対応付けがMarkdown記法(ルール)です

なお、どの意味(構造)に対してどのような見た目になるかはテーマ設定によって異なります(こちらはMarkdownの範囲外でCSSで調整します)

③ なぜ文章の意味や構造を表す必要があるのか？

答えとしては

「コンピュータにそれを読み取らせるため」です。

ただのベタテキストでは読み取れないし

それに応じた表示もできません

Markdownと表示のイメージ

スライド本文(2ページ目分のみ)

```
## 自己紹介
- 名前：ピカチュウ
- 所属：任天堂
- 趣味：**10まんボルト** ⚡ で世界を照らす

## 一言メッセージ
ピカピカ～！ ✨
```

内容は普通のテキストとして書かれています。

文章中に散在している `##` や `-` のような記号がMarkdown記法の「特定の構造を表すもの」として解釈されます

実際の表示

自己紹介

- 名前：ピカチュウ
- 所属：任天堂
- 趣味：10まんボルト ⚡ で世界を照らす

一言メッセージ

ピカピカ～！ ✨

Markdown記法から構造を読み取り、それを見た目に反映した表示になります

Markdownはあくまで文章の意味・構造を表すルールです。それらに対応した見た目の定義はテーマ設定やCSSといった別のテキストにより調整します

基本的な書式の説明は以上です
次章ではMarkdownの書式を
具体的に取り上げていきます

§ 4. Markdown

サンプルファイル

スライド生成の確認

03 フォルダ内のファイルを使用します

次のコマンドを実行します

```
marp markdown.md -o output.html
```

`output.html` が作成されるので開いて確認します

Markdown記法サンプル

作成日：2025-07-31

作成者：デモ用

このようなページが表示されればOKです
左右のカーソルキーでページが切り替えられます

今回のサンプルの位置付け

先ほどの `markdown.md` ですが、今回は直接中身を説明は致しません。

次ページ以降の解説事項を実際に手元で確認するための文字通りのサンプルとなります。

見出し

Markdown書式

見出し で見出しになります。

の数が見出しレベル(階層)です

```
# 見出し：レベル1
## 見出し：レベル2
### 見出し：レベル3
#### 見出し：レベル4
```

表示サンプル

サンプルファイルでは見出しは下記のようにになります

表示サンプル

見出し：レベル1

見出し：レベル2

見出し：レベル3

見出し：レベル4

強調・装飾・絵文字

Markdown書式

文字を特定の記号で囲うことで
強調や装飾ができます。
絵文字は普通に使えます

- ****太字****
- **斜体**
- ****太字と *斜体* の組み合わせ****
- ~~～打ち消し～~~
- 絵文字: 😊 🎉 ⚡

表示サンプル

- 太字
- 斜体
- 太字と 斜体 の組み合わせ
- 打ち消し
- 絵文字: 😊 🎉 ⚡

段落・改行

段落:Markdown書式

テキスト中に空行があると段落分けになります。

1行目:2行目との間に空行を入れない
2行目:1行目と同じ段落扱い

↑3行目は空行、4行目は1～2行めと別段落

表示サンプル

1行目:2行目との間に空行を入れない
2行目:1行目と同じ段落扱い

↑ 3行目は空行、4行目は1～2行目と別段落

改行:Markdown書式

テキスト上の改行はMarkdownでは無視されます。
行末に半角スペース2つを入れると Markdown上でも改行になります。(同じ段落扱い。その中で改行)

1行目:テキストエディタでは1行
2行目:Markdown上では1行目と同じ行の扱い
3行目:行末に半角スペース2つを入れる  
4行目:改行される(1～4行目まで同一段落)

6行目:空行挟んだので別段落

表示サンプル

1行目:テキストエディタでは1行 2行目:Markdown上では1行目と同じ行の扱い 3行目:行末に半角スペース2つを入れる
4行目:改行される(1～4行目まで同一段落)

6行目:空行挟んだので別段落

箇条書き

Markdown書式

- リスト項目 で箇条書きになります。
- の前に空白を入れると入れ子になります

```
- リスト項目1
- リスト項目2
  - ネスト項目A
  - ネスト項目B
```

表示サンプル

- リスト項目1
- リスト項目2
 - ネスト項目A
 - ネスト項目B

引用

Markdown書式

- > これは引用文です。
- > 複数行の引用も可能です。

表示サンプル

これは引用文です。複数行の引用も可能です。

コードブロック

インライン:Markdown書式

文章中にプログラムコードを記述する場合は ``` で囲みます。

表示サンプル

``Hello`` は `Hello` と表示されます

ブロック:Markdown書式

ブロック全体をコード表示する場合は ````` 3つで囲みます。後ろに言語を付けると色がつきます

```
```python
Pythonコード
def greet(name):
 print(f"Hello, {name}!")

greet("Pikachu")
```

### 表示サンプル

```
Pythonコード
def greet(name):
 print(f"Hello, {name}!")

greet("Pikachu")
```

# テーブル(表)

## Markdown書式

markdownで表を作る場合は `|` を使います。具体例は下記参照。  
(結構めんどくさいです)

## 表示サンプル

名前	種類	攻撃技
ピカチュウ	でんき	10まんボルト
フシギダネ	くさ／どく	つるのムチ
ヒトカゲ	ほのお	かえんほうしゃ

```
| 名前 | 種類 | 攻撃技 |
|-----|-----|-----|
| ピカチュウ | でんき | 10まんボルト |
| フシギダネ | くさ／どく | つるのムチ |
| ヒトカゲ | ほのお | かえんほうしゃ |
```

# リンク

## Markdown書式

Markdown(HTML)でのリンクとは、  
「クリックすると別の情報(ページやファイル)を開く“導線”」  
です。

書式は `[タイトル](URL)` となります  
URLはPDFなどのファイルでもOKです

```
[ゼルダの伝説](https://www.nintendo.com/jp/character/zelda/)
```

## 表示サンプル

[ゼルダの伝説](https://www.nintendo.com/jp/character/zelda/)

# 画像

## Markdown書式

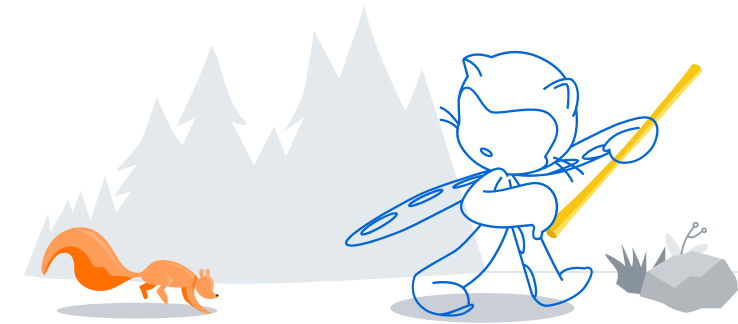
スライド内に画像を表示する場合は  
!`[タイトル](画像URL)` を使います。  
リンクとの違いは先頭の `!` の有無です

画像URLは外部のWEBサイトでも、  
ローカルの相対ファイルパスでもOKです

```
![GitHub](https://github.githubassets.com/assets/inbox-zero-86555dddc82e.svg)
```

※ 上記は表示の都合上改行されていますが  
本来は改行なしでの記述となります

## 表示サンプル



 応用（画像がリンクになってます）



```
[![GitHub](img/github-mark.png)](https://github.com/)
```

# HTML直接記述

## Markdown書式

markdownでは直接HTMLを記述することもできます

元々markdownはHTMLをシンプルに記述する為の省略記法で、ベースはHTMLそのものです。よってHTMLはそのまま記述できます (HTMLについては後述します)

```
<p>HTMLで<u>装飾</u>や
<strike>スタイル</strike>を指定</p>
```

## 表示サンプル

HTMLで装飾や ~~スタイル~~を指定



# 数式

## Markdown書式

数式はインライン表記とブロック表記の2種があります。インライン表記は文章中で数式部分を `$` で囲みます。

インライン: `$E = mc^2$`

ブロック表記は数式全体を `$$` で囲みます。

```
$$
\mathrm{i}\hbar \frac{\partial \psi}{\partial t}
= \hat{H}\psi(\mathbf{r},t)
$$
```

## 表示サンプル

インライン:  $E = mc^2$

ブロック表記:

$$i\hbar \frac{\partial \psi}{\partial t} = \hat{H}\psi(\mathbf{r},t)$$

## 数式自体の書式 → LaTeX

数式部分は `LaTeX` という記法を使います。これはmarkdownとは別の規格であり、HTML以前から存在する数式特化の記法です

↓ LaTeXサンプル

[Easy Copy MathJax](#)

# Marp独自機能

## Marp独自書式

markdownからは離れるのですが、marp独自の書式・機能があります  
ディレクティブと呼びます

[Marp公式の説明](#)

[Qiitaの解説記事](#)

```
<!-- _backgroundColor: #def -->
<!-- _header: ヘッダー -->
<!-- _footer: フッター -->
<!-- _paginate: false -->
```

## 表示サンプル

左ページのディレクティブを設定した場合の  
`output.html` の表示サンプルです

- 背景色が変わる
- 左上にヘッダー表示
- 左下にフッター表示
- 右下にページ表示

ヘッダー

### Marp独自機能（実践）

#### marp独自書式

markdownからは離れるのですが、marp独自の書式・機能があります  
ディレクティブと呼びます

[Marp公式の説明](#)  
[Qiitaの解説記事](#)

```
<!-- _backgroundColor: #def -->
<!-- _header: ヘッダー -->
<!-- _footer: フッター -->
<!-- _paginate: true -->
```

#### 表示サンプル

表示は次のページで確認です  
項目は次の4つです

- 背景色が変わる
- 左上にヘッダー表示
- 左下にフッター表示
- 右下にページ表示

14

フッター

# 改ページ(これもMarp独自機能)

## Markdown書式

Marpでは  で改ページになります

## の本来の定義

 が改ページというのはMarpの独自機能です。  
本来は水平線 `<hr>` を意味します。

↓ `<hr>` サンプル

## 表示サンプル

ここまでの改ページ全てです

## 見出しを改ページにする

Front Matterで下記を指定することで  
見出し( `# 見出し` )を改ページ & 見出しにできます

```
headingDivider: 2
```

※数字で改ページとなる見出しレベルを指定します



Markdownの説明は以上です  
次章ではMarkdownのベースである  
HTMLを取り上げます

## § 5. HTML

# HTMLとは

## HTMLとは

HTMLも中身はただのテキストファイルです。そして、Markdownと同様にテキストの構造を明示する記法が用意されています。

例えば、見出しはMarkdownの場合は

```
見出し
```

でしたが、HTMLの場合は

```
<h1>見出し</h1>
```

 となります。

上記のようにHTMLは基本的に `<タグ名>` と `</タグ名>` で対象の文字列を囲む形式です。これは「どこからどこまでがどの構造なのか」を明確にするための表記法です。

## HTMLとMarkdownの関係

Markdownは元々はHTMLをシンプルに記述する記法として作られました。

よって、お互いにどの表記がどの表記に対応するかという紐付けが可能です。(後ほど取り上げます)  
それではなぜHTMLを知る必要があるのか？

## HTMLを抑えるべき理由

MarkdownやHTMLは文章の構造を定義しますが、見た目は定義してません。

この見た目を制御するのはCSSという別の規格となります。そして、CSSはHTMLのどのタグに対してどのような見た目、という書式になっています。

よって、CSSで見た目をカスタマイズする為にはHTMLの理解が不可欠となります。

# Markdown ⇄ HTML対応表

MarkdownとHTMLの対応サンプルです。丸暗記対象ではないです。

Markdown	HTML	実例	備考
段落	段落	段落	-
# 見出し	見出し	(表示なし)	# の数で <h1> ~ <h6> に対応
**強調**	<strong>強調</strong>	強調	昔は <b> だったが今は非推奨
*斜体*	<em>斜体</em>	斜体	_斜体_ でも可
~打ち消し~	<del>打ち消し</del>	打ち消し	<strike> は非推奨
`コード`	<code>コード</code>	コード	インラインコード
[リンク](https://google.com)	<a href="https://google.com">リンク</a>	リンク	-
![代替テキスト](vim.gif)			-
—	<hr>	——	水平線。*** や ____ も可
<!-- コメント -->	<!-- コメント -->	(表示なし)	HTMLコメント。非表示

# Markdown ⇄ HTML対応

Markdown	HTML	実例				
<pre>```python print("hi")</pre>	<pre>&lt;pre&gt;   &lt;code&gt;     print("hi")   &lt;/code&gt; &lt;/pre&gt;</pre>	<u>コードブロック</u> <pre>print("hi")</pre>				
<pre>- リスト1 - リスト2</pre>	<pre>&lt;ul&gt;   &lt;li&gt;リスト1&lt;/li&gt;   &lt;li&gt;リスト2&lt;/li&gt; &lt;/ul&gt;</pre>	<u>箇条書き</u>    - リスト1   - リスト2				
<pre>&gt; 引用</pre>	<pre>&lt;blockquote&gt; 引用 &lt;/blockquote&gt;</pre>	<u>引用</u>  引用				
<pre>  見出し1   見出し2      —     —      本文1   本文2  </pre>	<pre>&lt;table&gt;   &lt;th&gt;見出し1&lt;/th&gt;   &lt;th&gt;見出し2&lt;/th&gt;   &lt;tr&gt;     &lt;td&gt;本文1&lt;/td&gt;     &lt;td&gt;本文2&lt;/td&gt;   &lt;/tr&gt; &lt;/table&gt;</pre>	<u>表</u> <table><tr><th>見出し1</th><th>見出し2</th></tr><tr><td>本文1</td><td>本文2</td></tr></table>	見出し1	見出し2	本文1	本文2
見出し1	見出し2					
本文1	本文2					

37/



# タグのクラス分け

## なぜクラス分けが必要か

同じタグを使った文章であっても、異なる見た目にしたい場合があります。

例えば、`**強調**` はデフォルトでは 強調(太字のみ) ですが、**強調(太字+マーカー)** という見た目にしたい場合があります。

そういった**仕分け用の追加情報としてクラス**があり、タグ毎に設定することができます。

## クラスの設定

**クラスはHTMLタグに対して設定**します。  
Markdownの省略記法では設定できません。

書式例は下記です。

```
<strong class="クラス名">強調
```

## 書式例の解説

```
<strong class="クラス名">強調
```

`strong` はタグ名です。`p` でも `h1` でもどのタグでも同じ書き方になります。`class="クラス名"` **がクラス名の設定**部分です。クラス名は自由に決められます。

尚、クラスを設定しても、そのクラスに対する何かしらの見た目(CSS)や動作(JavaScript)の設定がない場合はデフォルトと何ら違いはありません。

## サンプル

```
強調
```

強調

```
<strong class='mark-yellow'>強調
```

**強調** ← CSSの設定があるので見た目が変わる 38/63

# タグのID

## なぜIDが必要か

IDもクラスと同様に「仕分け用の追加情報」です。

クラスとの違いは「**IDは1つの文書(HTML)において唯一1つだけ**」という決まりがある点です。

用途としては、リンク先をHTML文書内の特定の箇所として指定する場合や、JavaScriptなどで操作対象をピンポイント指定する場合などです

**Marpではページ指定**に使用されています

## IDの設定

IDもHTMLタグに対して設定します。書式例は下記

```
<h2 id="ID">見出し</h2>
```

## 書式の解説

```
<h2 id="ID">見出し</h2>
```

`h2` はタグ名です。`p` でも `h1` でもどのタグでも同じ書き方です。`id="ID"` が**IDの設定**部分です。IDは自由に決められます。ページ数などでもOKです

## サンプル

先の `output.html` でページ数を指定する場合

```
[sample3](sample/03/output.html)
```

[sample3](#) ← ファイルのみ指定。最初のページが表示

```
[sample3#5](sample/03/output.html#5)
```

[sample3#5](#) ← #5でID=5を指定。p5が表示



# <div>タグ, <span>タグ

## クラスやID設定で困ること

先に紹介したクラス & IDですが、「何らかのタグに付随して設定する」必要があるため、「**クラス/IDだけを設定したいのにタグに引きずられて見た目が変わる**」という問題が生じます。

見た目だけでなく、JavaScriptなどで制御するための文章やデータとしての構造を明示したい場合など、**タグが前提にあることで余計な意味が付いてきて困ることがあります**。

## div, spanの意義

そこで、**見た目への影響が一切なく、構造的な意味も一切持たないタグ**が用意されています。

それがdivとspanです。

## 書式&サンプル

書式は下記の通りで一般のタグの場合と同じです

```
<div class='box-info'>囲み枠にしたい文章</div>
人物名
```

1行目の実例が下記です。`class='box-info'` に対して囲み枠となるように見た目を定義しています。(そして他タグからの余分な影響を受けていません)

囲み枠にしたい文章

## divとspanの使い分け

divはブロック、spanはインライン特化です。

イメージとしてはdivはどこでも汎用的に使えて、spanは文章中の特定文字列に適用、という感じです



HTMLの説明は以上です。冒頭で  
「CSSで見た目をカスタマイズする為には  
HTMLの理解が不可欠」  
と述べたとおり  
次章ではCSSを取り上げます

## § 6. CSS 📄

どうでもいいお話ですが、CSS 📄、HTML 📄、Markdown 📄といった風に  
フォーマットや規格を表すアイコンとかも結構あります

# サンプルファイル

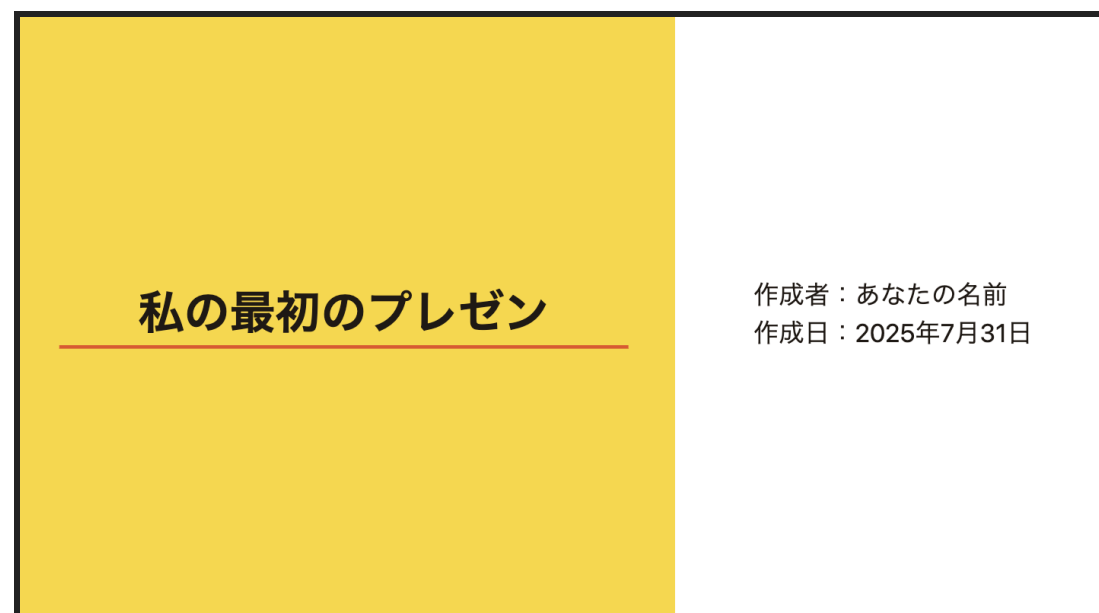
## スライド生成の確認

04 フォルダ内のファイルを使用します

次のコマンドを実行します。 `--theme` がCSSの指定です

```
marp --theme sample.css slide.md -o output.html
```

`output.html` が作成されるので開いて確認します



このようなページが表示されればOKです  
左右のカーソルキーでページが切り替えられます

## 今回のサンプルの位置付け

今回のサンプルは

- **CSSのカスタマイズでどれだけ見た目を変化するか**の例示
- 実際にどういった設定を行ったのかの説明との対応付け

が目的です。

# 見た目をカスタマイズするための下準備

## 見た目カスタマイズ=CSS

CSSとはHTMLやMarkdownで記述された文書の見た目(デザイン)を定義・カスタマイズする規格です

本章ではこのCSSファイルについて説明します

## Markdownの追記

FrontMatter部分に `theme:` 項目を追記します

```
theme: custom
```

`custom` は右ページのCSSのファイルに記載するテーマ名と一致させる必要があります

## CSSファイルを作成

CSSファイルも中身はテキストファイルです。普通にメモ帳などのテキストエディタで作成します。

ファイル名は `custom.css` とし下記2行を記述します

```
/* @theme custom */
@import default;
```

**i** 本設定はMarp独自の設定です(一般のHTMLでは不要)

## 動作確認

次のコマンドを実行し、`output.html` を確認します

```
marp --theme custom.css slide.md -o output.html
```

## 私の最初のプレゼン

作成者：あなたの名前

作成日：2025年7月31日

# 最初のカスタマイズ(背景色)

## コード

`custom.css` の末尾に次のコードを追記します

```
/* 全体 */
section {
 background: #fed823;
 color: #241915;
}
```

## 動作確認

次のコマンドを実行し、`output.html` を確認します

```
marp --theme custom.css slide.md -o output.html
```

### 私の最初のプレゼン

作成者：あなたの名前  
作成日：2025年7月31日

## 説明(簡易)

- `/* 全体 */` はコメントです。動作には影響しません
- `section {...}` はスライド全体の設定です
- `background: #fed823;` は背景色の設定
- `color: #241915;` は文字色の設定

🔗 上記理解の前提となるCSSフォーマットの説明を次ページから行います

📌 以後、動作確認の記載は省略します



# セレクタ・プロパティ・値

## コード(再掲)

```
/* 全体 */
section {
 background: #fed823;
 color: #241915;
}
```

## セレクタ

上記CSSコードについての見方を説明します

`section` の箇所をセレクタと呼びます。**セレクタは何に対してカスタマイズを適用するか**を表します。具体的にはタグ( `h1` や `p` など)やクラス、IDを指定します。`section` はスライド全体を表します

セレクタに続く `{ ... }` は指定したセレクタに対してのカスタマイズ内容を記述します。見た目のカスタマイズ・デザイン設定のことをスタイルと呼びます。  
`{ ... }` は**スタイルの記述場所**です

## スタイルの書式: プロパティと値

`{ ... }` 内の記述の仕方は下記の通りです。

プロパティ: 値;

**プロパティはスタイルのどの要素を設定するか**を表します。`background` なら背景色、`color` は文字色です

**値はプロパティに対する具体的な設定内容・値**です

プロパティと値の間にはコロン `:`、値の後にはセミコロン `;` を記述します

≡ 先の `background: #fed823;` は  
`background` がプロパティ、`#fed823` が値です

## コメント

`/* ... */` は**コメント**です。表示にも動作にも影響しません。ただしMarpでは一部特殊機能があります

# 具体例(2ページ目)

## コード

custom.css に次のコードを追記します

```
/* 2ページ目 */
h2 {
 /* ***** 1.ボックス設定 ***** */
 /* 大きさ */
 width: 80%;
 /* 余白 */
 margin: 0.5em auto 0.5em auto;
 padding: 0.5em 0.5em 0.5em 0.5em;
 /* padding装飾 */
 background: #251915;
 /* border */
 border: solid 8px #ea4f24;
 /* ***** 2.コンテンツ領域 ***** */
 /* 文字配置 */
 text-align: center;
 /* 文字装飾 */
 font-size: 1.8em;
 font-weight: bold;
 color: #ffffff;
}
```

**i** どれがセクタ・プロパティ・値かが分かればOKです

## 見た目の変化

### before

#### 自己紹介

- 名前：ピカチュウ
- 所属：任天堂
- 趣味：10まんボルト ⚡ で世界を照らす  
(そしてちょっと焦がす)

#### 一言メッセージ

ピカピカ〜! ⚡

### after

#### 自己紹介

- 名前：ピカチュウ
- 所属：任天堂
- 趣味：10まんボルト ⚡ で世界を照らす  
(そしてちょっと焦がす)

#### 一言メッセージ

ピカピカ〜! ⚡

**□** 注目するのは「変化」です。センスじゃないです



# ボックスモデル

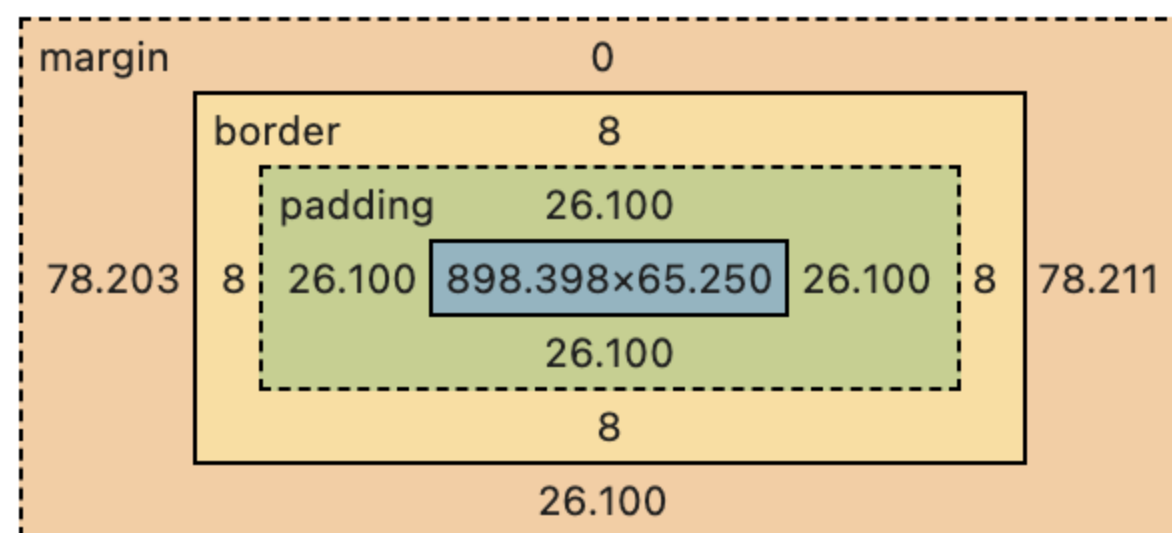
## ボックスモデルとは

前ページの `h2` に対するスタイルを読み解くには、ボックスモデルというアイデアの理解が必要になります

ボックスモデルとはコンテンツ(文字列)を一つのボックス(長方形)と見なし、更に余白となるボックスで囲むことでコンテンツと余白をまとめて一塊りのボックスとして表現する方法です

## ボックスモデル全体図

まず全体図を提示します。各構成要素を順次説明します



## コンテンツ領域

中央にある下記部分をコンテンツ領域と呼びます

898.398×65.250

コンテンツ領域とは文字列や画像といった具体的な中身を囲むボックスです

## padding領域, margin領域

paddingとmarginはコンテンツ領域の周りに余白をつける囲みです。paddingには背景色や枠線といった装飾が適用できます。marginは基本的には余白のみです。marginまでを含めて1つのボックスとして扱われます

## border領域

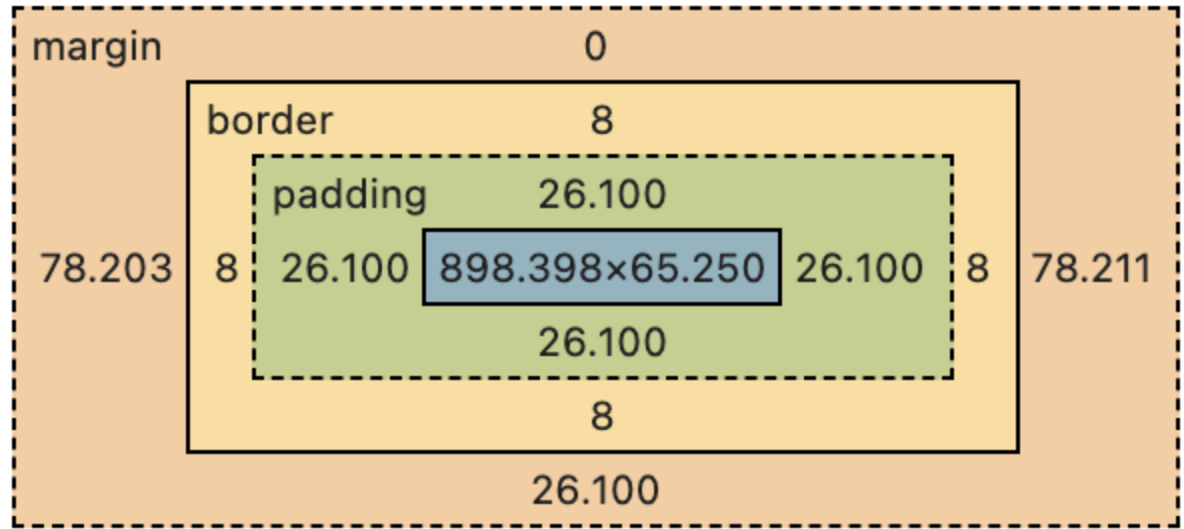
borderはpaddingの周りにつける枠線です。余白全体はpadding + border + marginとなります



↑ 手元のブラウザで確認する手順

# ボックスモデルとしてh2を見る

## ボックスモデル全体図(再掲)



## 2ページ目(再掲)

自己紹介

- 名前：ピカチュウ
- 所属：任天堂
- 趣味：10まんボルト で世界を照らす  
(そしてちょっと焦がす)

一言メッセージ

ピカピカ～！

## h2におけるボックス

- 自己紹介

↑ 実際の表示です
- 自己紹介

↑ 緑のボックスがコンテンツ領域です。  
幅は `width` で指定、高さは自動調整です
- 自己紹介

↑ paddingです。コンテンツを囲っています  
コンテンツ領域と共通の背景色が適用されます
- 自己紹介

↑ borderです。分かりにくいですが、paddingの周りの枠線です
- 自己紹介

↑ marginです。borderの更に外です  
このボックスがh2全体を表します

# ボックスのサイズ設定

## コンテンツ領域



コンテンツ領域は中心となるボックス(長方形)のため、幅と高さの情報が必要になります。

いずれもデフォルトでは自動的に最小限の大きさに設定されるので、一つ一つ指定する必要はありません。

ただし、手動で設定したい場合(上記のh2のように幅を大きくしたい、等の場合)は CSSにて下記のように指定します

```
h2 {
 width: 80%; /* 幅 */
 height: 100px; /* 高さ */
}
```

width は幅、height は高さを指定するプロパティです

## 大きさを表す単位

幅を表す 80% や高さを表す 100px のように数値の後に単位がついています。これらの単位の意味は次のようになります

単位	意味	解説
px	ピクセル	PCの表示画面においての何ピクセルかを表します
%	パーセント	親要素に対する割合を表します
em	何文字分	何文字分の大きさかを表します

他にも rem やら vh やら pt やら色々ありますが、それらは後にして、まずは上記を抑えてください

💡 数値が0の場合のみ単位を省略することができます

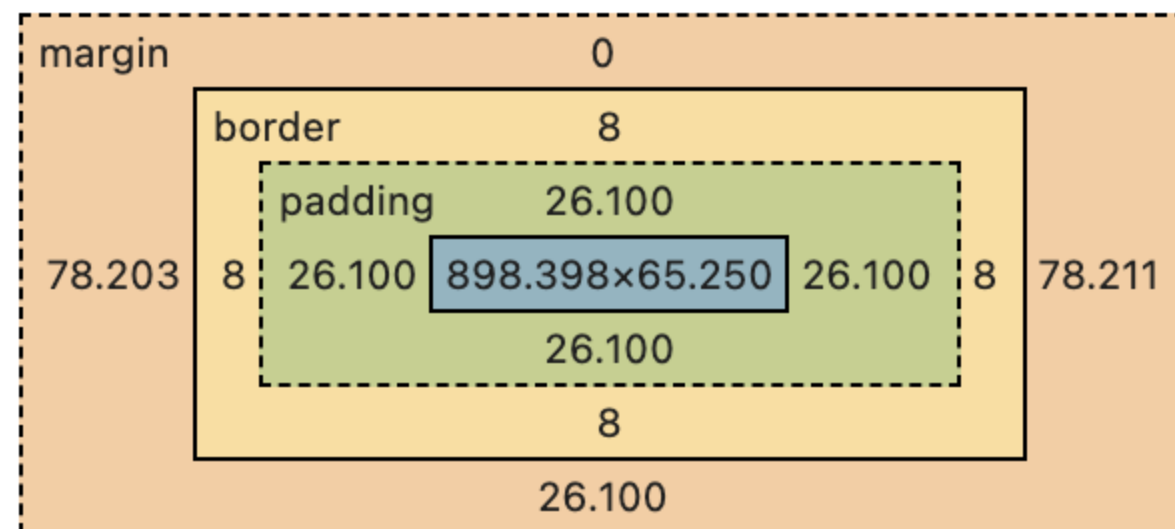
# padding領域

## paddingの範囲

下図の緑部分がpadding領域の範囲になります



paddingはコンテンツ領域の周りを余白として囲う形になるので、幅・高さではなく、上下左右の隙間の大きさを表す情報が必要になります



上記の図における padding の部分が該当します。上図では上下左右とも 26.100 です

## CSS

paddingはCSSにて下記のように指定します

```
h2 {
 /* 余白: 上 右 下 左 */
 padding: 0.5em 0.5em 0.5em 0.5em;
}
```

順番は固定で上→右→下→左です。ただし、数値の個数により対象箇所が変わります。

```
/* 余白: 上 左右 下 */
padding: 0.5em 0.5em 0.5em;
/* 余白: 上下 左右 */
padding: 0.5em 0.5em;
/* 余白: 上下左右 */
padding: 0.5em;
```

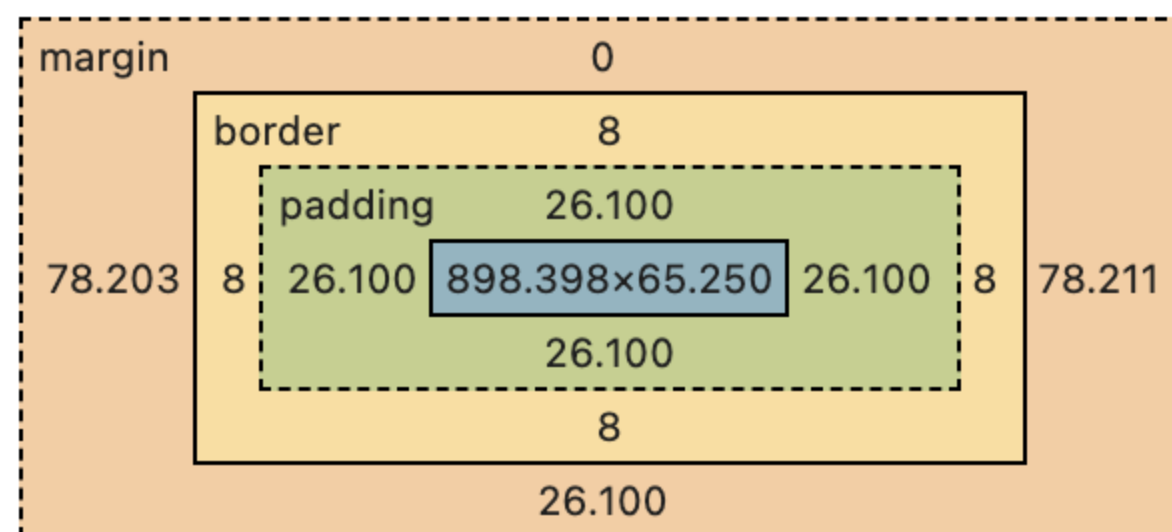
# margin領域

## marginの範囲

下図の緑部分がmargin領域の範囲になります



`margin` は `コンテンツ+padding+border領域` の更に外側を余白として囲います。`padding` と同様に上下左右の隙間の大きさを指定します



上記の図における一番外の `margin` の部分が該当します。数値は `0 → 78.211 → 26.100 → 78.203` です

## CSS

`margin`はCSSにて下記のように指定します

```
h2 {
 /* 余白: 上 右 下 左 */
 margin: 0.5em auto 0.5em auto;
}
```

順番は固定で上→右→下→左です。ただし、数値の個数により対象箇所が変わります。

```
/* 余白: 上 左右 下 */
margin: 0.5em auto 0.5em;

/* 余白: 上下 左右 */
margin: 0.5em auto;
```

`auto` は自動で中央に配置されるように設定されます

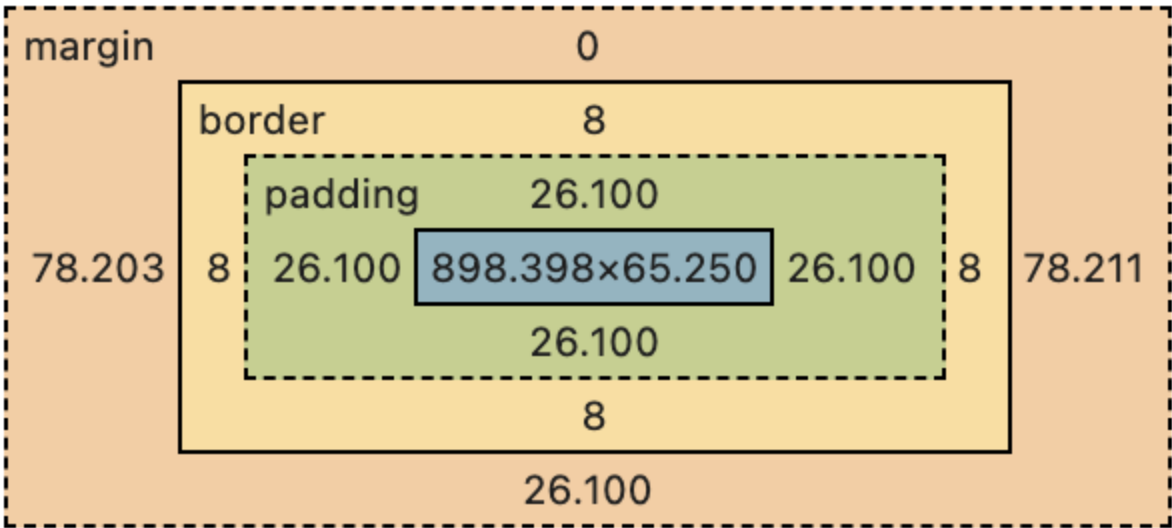
# border領域 = 枠線

## borderの範囲

下図の緑部分がborder領域の範囲になります



`border` は `コンテンツ + padding領域` の外側に主に装飾的な枠線として設定します。枠線の太さと色や線の種類を指定します



上記の図における `border` の部分が該当します。幅はいずれも `8` です

## CSS

`border`はCSSにて下記のように指定します

```
h2 {
 /* 枠線: 形状 太さ 色 */
 border: solid 8px #ea4f24;
}
```

💡 ``border``は``padding``や``margin``とは異なり、装飾的な意味合いが強いです。そのため、設定のプロパティも値も種類が多く、必要に応じて右上のリンク先を参照ください。



ボックスモデルの説明は以上です  
続いて  
コンテンツ領域の中身のCSSスタイル  
を説明します

# コンテンツ領域の中身

## コンテンツ領域の中身とは

コンテンツ領域の中身 = 文字(文章)・画像などです。  
今回は文字について取り上げます

## 実際の見たい目

自己紹介

## CSSコード

```
h2 {
 /* ***** 2.コンテンツ領域 ***** */
 /* 文字配置 */
 text-align: center;
 /* 文字装飾 */
 font-size: 1.8em;
 font-weight: bold;
 color: #ffffff;
}
```

## 説明

CSSコード内にコメントしているように文字の配置  
(中央寄せ、左寄せなど)や文字の装飾(大きさ、太さ、  
色)を設定しています

- `text-align: center;`
  - 中央寄せにしています
- `font-size: 1.8em;`
  - 文字サイズをデフォルトの1.8倍にしています
- `font-weight: bold;`
  - 太字
- `color: #ffffff;`
  - 文字色を白にしています



# 2ページ目のその他の設定

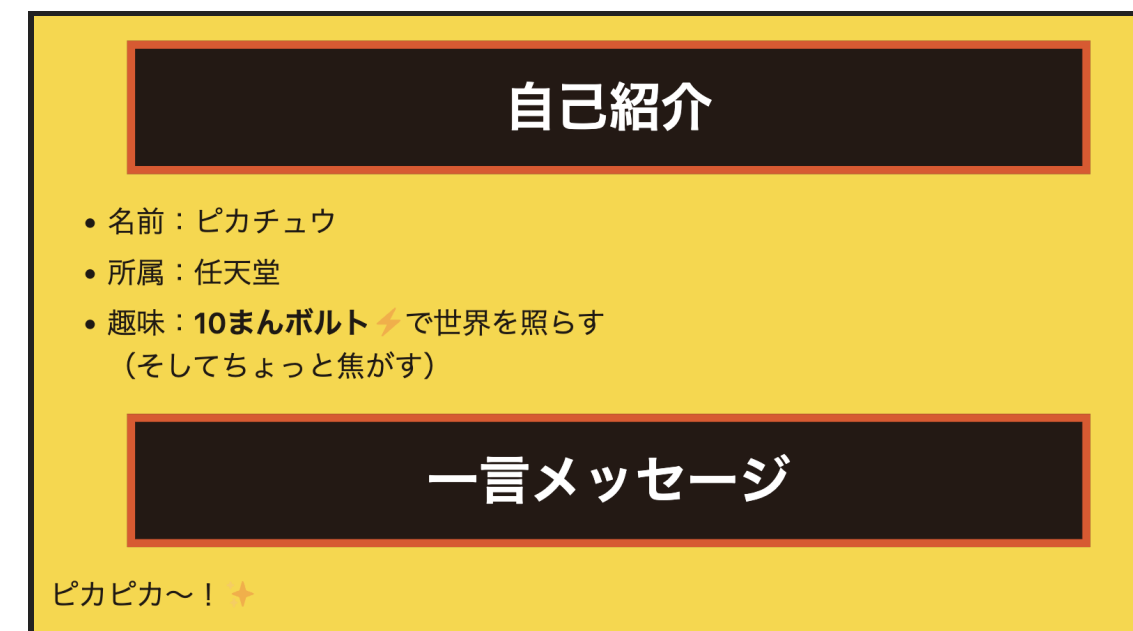
## CSSと設定内容

`p` , `ul` は幅とマージンを設定し中央揃え表示します。`strong` は強調文字の色設定です

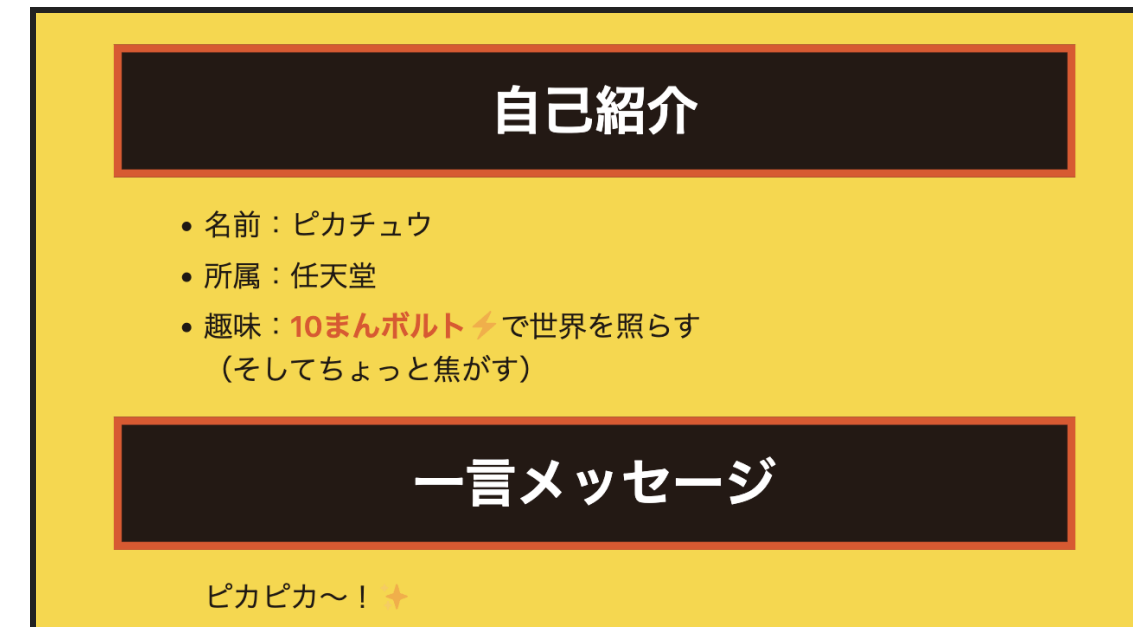
```
p, ul {
 width: 70%;
 margin: 0.5em auto;
 padding: 0;
}

strong {
 color: #ea4f24;
}
```

## before



## after





# 1ページ目設定

## CSS

```
section.title { background: linear-gradient(90deg, #fed823 0%, #fed823 60%, #ffffff 60%, #ffffff 100%); }
h1 {
 /* サイズ&位置 */
 width: 50%;
 position: absolute;
 top: 50%;
 left: 5%;
 transform: translate(0%, -50%);
 /* 余白&枠線 */
 margin: 0em;
 padding: 0.1em;
 border-bottom: solid 4px #ea4f24;
 /* 文字配置&装飾 */
 text-align: center;
 font-size: 1.8em;
 font-weight: bold;
 color: #241915;
}
section.title p {
 width: 30%; /* サイズ */
 position: absolute; /* 位置 */
 top: 50%;
 left: 67%;
 transform: translate(0%, -50%);
 margin: 0px; /* 余白 */
}
```

## セレクトタの補足

`section.title` は `section` タグで且つクラスが `title` という意味になります。`section` はスライド全体を指します。クラス `title` は `slide.md` の下記にて設定しています。(Marp独自の記法です)

```
私の最初のプレゼン
<!-- _class: title -->
```

## プロパティ&値の補足

`linear-gradient( ... )` は色指定の方法の一つで、複数の色のグラデーションを指定します。今回は左右2分割にしています

`transform: ...` はCSSのボックスを変形します。今回は `translate` を使って左右の位置を調整しています

CSSの説明は以上です  
次はMarpの応用的な使い方を説明します

## § 7. Marpの応用的な使い方

# テキストベースを活かす

## ファイル分割

スライドの元になるファイルがテキストファイル (Markdown) のため、複数ファイルへの分割が可能です。

```
000_init .md
001_loading-htmlonly .md
010_表紙 .md
100_学習理由...標、進め方 .md
200_導入 .md
300_書式 .md
320_markdown .md
330_HTML .md
340_CSS .md
800_応用 .md
script-htmlonly .md
```

スライド生成時は前処理的にPythonやシェルスクリプトで1つに結合します。

## バージョン管理

テキストファイルということは、プログラムのソースコードと同様に GitHubなどを用いてバージョン管理・変更管理が可能です。

==MarkdownやHTMLは文章の構造を定義しますが、見た目は定義してません。==

この見た目を制御するのはCSSという別の規格となります。

-  
そして、CSSはHTMLのどのタグに対してどのような見た目、という書式になっています。

+そして、CSSは==HTMLのどのタグに対して==どのような見た目、という書式になっています。

# HTML&CSSを活かす

## 小技: WEBフォントが使える

PCにインストールされていないフォントでも利用できます。下記は[Google Fonts](https://fonts.google.com/)を利用しています

```
@import url("https://fonts.googleapis.com/css2?family=BIZ+UDGothic&family=BIZ+UDPGothic:wght@400;700&display=swap");

section {
 font-family: "BIZ UDPGothic", sans-serif;
}
```

## 小技: 文字を詰める

文字間を詰めることが可能です

```
.text-spacing { letter-spacing: -0.04em; }
```

### before(1文字はみ出てる)

※バージョン名は最新版に合わせて読み替えてください

### after(1行に収まってる)

※バージョン名は最新版に合わせて読み替えてください



# Node.js ベースを活かす

## Node.jsとは

**Node.jsはJavaScriptの実行環境**です。Marp本体もNode.js上で実装されています(本レクではNode.jsは使わず、バイナリアプリを利用しています)

## markdown-itのplugin利用

MarkdownをHTMLに変換する機能はNode.jsのmarkdown-itというライブラリを使っています。よって、pluginを追加設定することで**標準で対応していない表記などが利用可能**になります

```
"dependencies": {
 "@marp-team/marp-cli": "^4.2.0",
 "@md-reader/markdown-it-mermaid",
 "markdown-it-anchor": "^9.2.0",
 "markdown-it-attrs": "^4.3.1",
 "markdown-it-cjk-breaks": "^2.0.
```

例: ルビを振る → よこみちにそれました 閑話休題

## Dockerによるコンテナ化

機能拡張した状態でDockerによるコンテナ化が可能です。これにより、**OSや環境に依存せずMarpを利用**することができます(WindowとMacを両方使っている場合でもMarpの動作を統一できます)

% docker images		IMAGE ID	CREATED	SIZE
REPOSITORY	TAG			
my-marp	latest	8d32f098e17b	12 days ago	1.28GB

 この辺のレク欲しい人はアンケートお願いします

# おしまい

以上で終了です  
初見の方には多種の知識が求められ大変だったかも知れませんが  
何度か作成してる内に慣れてくると思います  
実践大事